

Informe proyecto final

Norma Constanza Giraldo Reyes

Hernando David Mosquera Jiménez

Facultad de Ciencias e Ingeniería, Universidad Manizales

Programación 1

Natalia Marcela Castellanos Gómez

17 de mayo de 2026

Introducción:

El presente informe describe el desarrollo de un sistema de gestión de tareas implementado en Python mediante el paradigma de programación orientada a objetos. Esta solución aborda la necesidad de organizar, priorizar y dar seguimiento a tareas en entornos colaborativos, en los que la ausencia de una herramienta centralizada puede provocar la pérdida de información y retrasos en el cumplimiento de plazos.

El documento presenta el planteamiento del problema, el caso de uso del sistema, las funcionalidades implementadas, la estructura de clases diseñada y los principios de Programación Orientada a Objetos aplicados. Finalmente, se exponen las conclusiones obtenidas a partir del proceso de desarrollo.

Planteamiento del problema:

En los entornos laborales y académicos actuales, la gestión eficiente de actividades constituye uno de los mayores desafíos para individuos, equipos y organizaciones. La ausencia de un sistema centralizado para registrar, priorizar y hacer seguimiento de tareas genera pérdida de información, incumplimiento de fechas de entrega, duplicidad de esfuerzos y dificultad para identificar el estado real de los proyectos en curso.

Las herramientas tradicionales como listas en papel, hojas de cálculo o correos electrónicos no ofrecen la flexibilidad ni la estructura necesaria para administrar tareas con múltiples niveles de complejidad, asignaciones a diferentes responsables, categorización por prioridad o la división de una tarea en subtareas más pequeñas y manejables.

Adicionalmente, cuando múltiples usuarios deben colaborar dentro de un mismo proyecto, se hace imprescindible contar con un mecanismo que permita notificar a los involucrados sobre cambios, vencimientos o actualizaciones importantes, ya sea a través de correo electrónico o mensajes SMS, garantizando así la comunicación oportuna.

Caso de uso: Sistema de gestión de tareas / To-Do List:

Descripción:

El Sistema de Gestión de Tareas es una aplicación de consola desarrollada en Python bajo el paradigma de Programación Orientada a Objetos (POO). Su propósito principal es permitir a usuarios y clientes organizar, priorizar y hacer seguimiento de proyectos y sus tareas asociadas, facilitando la colaboración entre equipos de trabajo y el cumplimiento de plazos establecidos.

El sistema contempla la gestión completa del ciclo de vida de un proyecto: desde el registro de los actores involucrados (usuarios y clientes), pasando por la creación y seguimiento de proyectos, hasta la administración granular de tareas y subtareas, con soporte de notificaciones automáticas por correo electrónico y SMS.

Roles del sistema:

Rol	Descripción
Usuario	Miembro del equipo de trabajo. Puede ser asignado como líder de proyecto o responsable de tareas/subtareas. Posee nombre, apellido, correo, cargo y fecha de nacimiento.
Cliente	Empresa o persona que solicita el proyecto. Tiene nombre de empresa, nombre del contacto principal, correo, teléfono y área empresarial.

Funcionalidades principales

a) Gestión de usuarios

- Registro de usuarios con nombre, apellido, correo electrónico, cargo y fecha de nacimiento.
- Validación automática del formato del correo electrónico al momento del registro.
- Cálculo automático de la edad del usuario a partir de su fecha de nacimiento.
- Consulta del listado completo de usuarios registrados en el sistema.

b) Gestión de clientes

- Registro de clientes empresariales con datos de contacto completos.
- Búsqueda de clientes por correo electrónico o número de teléfono.
- Asociación de un cliente como dueño de uno o múltiples proyectos.

c) Gestión de proyectos

- Creación de proyectos con título, descripción, fechas de inicio y fin.
- Asignación de un cliente y un líder de proyecto (usuario) a cada proyecto.
- Consulta del listado de todos los proyectos activos.

d) Gestión de tareas

- Creación de tareas con identificador único autogenerated, título, descripción, categoría, prioridad y estado.
- Asignación de una fecha de entrega con validación de que sea una fecha futura.
- Asociación de cada tarea a un proyecto específico y a un usuario responsable.
- Adición de comentarios a las tareas para registrar avances o aclaraciones.
- Visualización detallada de todas las tareas o filtradas por proyecto.

e) Gestión de subtareas

- Creación de subtareas que heredan todos los atributos de una tarea principal.
- Referencia explícita a la tarea principal a la que pertenece cada subtask.
- Visualización de subtareas con identificación de su tarea padre.

f) Sistema de notificaciones

- Envío de notificaciones con asunto, descripción, detalle y marca de tiempo.
- Soporte de dos canales de notificación mediante polimorfismo: Notificación por correo electrónico (EmailNotification) y Notificación por SMS (SmsNotification).
- La clase base Notificación es abstracta, garantizando que cada canal implemente su propio método de envío (send()).

Estructura del sistema:

El sistema fue diseñado aplicando los principios fundamentales de la POO. La siguiente tabla resume la jerarquía de clases implementada:

Clase	Tipo / Relación	Responsabilidad principal
Usuario	Clase base	Representa a los miembros del equipo con validación de datos personales.
Cliente	Clase independiente	Modela la empresa o persona que contrata el proyecto.
Proyecto	Composición (usa Usuario y Cliente)	Agrupar tareas bajo un contexto y responsable definido.
Tarea	Composición (usa Usuario y Proyecto)	Unidad de trabajo con estado, prioridad, categoría y fecha límite.
SubTarea	Herencia (extiende Tarea)	Subdivisión de una tarea principal; hereda todos sus atributos.
Notificación	<i>Clase abstracta</i>	Define el contrato común para todos los canales de notificación.
EmailNotification	Herencia (extiende Notificación)	Implementa el envío de alertas por correo electrónico.
SmsNotification	Herencia (extiende Notificación)	Implementa el envío de alertas por mensaje SMS.

Principios de POO aplicados:

- **Encapsulamiento:** Todos los atributos de cada clase son privados (prefijo _). El acceso y modificación se realiza exclusivamente a través de métodos getter y setter públicos, protegiendo la integridad de los datos.
- **Herencia:** SubTarea extiende Tarea, reutilizando todos sus atributos y métodos y agregando la referencia a la tarea padre. EmailNotification y SmsNotification extienden la clase abstracta Notificación.
- **Polimorfismo:** La clase Notificación define el método abstracto send(). Cada subclase lo implementa de forma distinta según el canal de envío, permitiendo tratar objetos de distintos tipos de forma uniforme.

- **Abstracción:** Notificación es una clase abstracta (ABC) que no puede instanciarse directamente. Define el contrato que deben cumplir todas las notificaciones del sistema.

Flujo principal de uso:

1. El administrador registra los usuarios que participarán en los proyectos.
2. Se registra el cliente que solicita el trabajo.
3. Se crea un proyecto asociándolo al cliente y designando un líder (usuario).
4. Se crean tareas dentro del proyecto, asignando responsable, prioridad, categoría y fecha límite.
5. Las tareas complejas se subdividen en subtareas para un mejor seguimiento.
6. Se consulta el estado de las tareas por proyecto para monitorear el avance.
7. El sistema envía notificaciones a los involucrados a través del canal configurado (correo o SMS).

Conclusión:

El desarrollo del Sistema de Gestión de Tareas permitió aplicar de manera práctica los fundamentos de la **Programación Orientada a Objetos**, consolidando conceptos como encapsulamiento, herencia, polimorfismo y abstracción en un proyecto funcional y con sentido real. A través de la construcción de clases como Tarea, SubTarea, Notificación y sus derivadas, se comprendió cómo modelar problemas del mundo real en estructuras de código organizadas, reutilizables y fáciles de mantener. Este proyecto demostró que un buen diseño orientado a objetos no solo hace el código más limpio, sino que también facilita su crecimiento y adaptación ante nuevos requerimientos.